

Software Architecture

Availability and Services

11

Contents

- Components vs. Services
- Monitoring Availability
- Impact of Downtime
- Control Cascading Failures: Circuit Breaker, Canary Call
- Redundancy with Replication: Consistency vs. Availability
- Event Sourcing

After discussing about how can we scale, let's look into the availability of the architecture. And as we have been looking at different qualities, we have been trying to find what is the construct, for example, the architectural concepts (e.g., components and modularity, or interface and reusability and so on) that are conceptually close with the corresponding quality attribute.

Today we will make the jump between software components and services. We will use the difference between components services to define the challenge of availability. When we look into the quality of flexibility and how can the architecture deal with change, we will make the next jump going from services to microservices.

As part of the lecture, we're going to give an operational definition of availability. What does availability imply? and what is the impact if the system is not available? What could possibly go wrong if we have a downtime? This is particularly critical for services. We will show how we can monitor it and how to design systems using patterns that can help us to design systems that are highly available. In the same way that we looked into how to scale and learn how to design a scalable architecture, we are going to look into how do we design highly available architectures.

Example

Components vs. Services

Map Drawing Component

```
//lat, lng - any GPS coordinate on Earth  
//zoom - show details up to 1m resolution  
Map getMap(lat, lng, zoom)
```

- What is the "size" of this component?
- How to deliver the component to customers so that it can be included in their own applications?

To introduce the idea of what is a service and what is the difference between a service and a component, I would like to present you with this simple component interface.

The component purpose is to be able to display a geographical map. In the interface there is a single operation which receives GPS coordinates anywhere on the planet, and then it has also a zoom level. The result is a map. You can see it as an image, or some other visual representation.

Knowing that you can show a very detailed map, up to 1 meter resolution, or you can zoom out and just show the borders between the different countries, what I would like to ask you is: if you have to build the component and then you have to deliver it to your customers which will take the component and for example use it to display a map in their application – What is the size of this component?

We know that we have to deploy components before we can use them in production and to deploy them we have to package them so they can be released. And if we have to make them self-contained so that there are no dependencies, all the software and all of the data (the most important element in this case) that is necessary for the component to work will be included in the package.

Do you have an estimate for how big is this component going to be? How many bits?

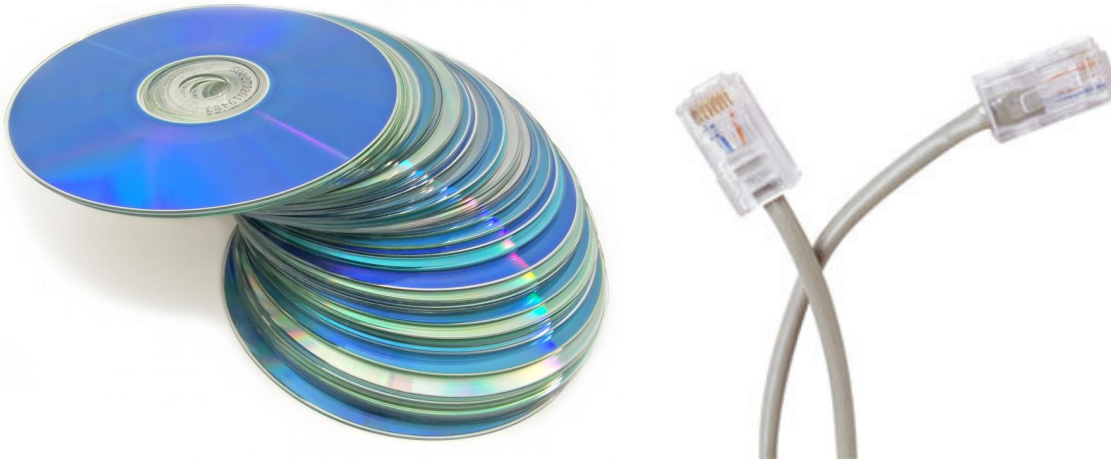
If you try to go back in time, from your personal experience, do you remember a time in the history of software architecture in which the Internet didn't exist? All software had to work offline.

There used to be a time in which if you had to package and release the component, you had to actually copy it from a golden master on some physical media: Floppy disks, CDs, DVDs, USB sticks. Clearly, you are limited by the capacity of the media.

If you have to deliver the map component whose size is 7 terabytes, how many discs do you need? How much is that going to cost you?

Example

Components vs. Services



The software industry before the Internet looked like this. We were writing code, but there was a physical aspect to it at the end of the build process you had to burn the software into CDs. Imagine how much plastic e-waste was produced with every upgrade to deliver the latest version, how much energy to ship those disks around the world. Also, someone had to physically unwrap them and use them to load the software onto each piece of hardware to deploy the upgrade.

One problem with maps is that they tend to change. New houses are built, new roads and bridges are built, and the map tends to become obsolete over time. We need a stream of CDs that you ship whenever the map needs to be updated.

After the Internet came into existence, people started to find ways for the software to be more efficiently delivered. You can take an image of the CD and you can make it available as a file and then you can put it on a website or an FTP server. Users can connect their computers to the FTP server, download the image of the software and install it.

This was greatly streamlining the software delivery experience. There was no longer the "commercial off the shelf" (COTS) software metaphor, because there was no shelf from which you would take a beautifully designed box to open the box and get the CD out of it. Now you just had to find out the proper address of the website. And maybe after exchanging some payment, you would download and install the app.

After many attempts, App stores were born and you had a huge selection of software. But this was still about finding an efficient mechanism for transferring the software. The bits to unpack and install from their source into the environment in which they get deployed into so that they can run.

Another opportunity for delivering software through the Internet is actually to switch to a distributed architecture. A client/server style in which your software is not being delivered at all. It runs somewhere else, and if you want to use it, you're just sending messages to it and getting results and interacting with it from a client.

In this case you have the option of whether you have to install a specific client. You solve the problem in half: the majority of the software, for example, the data, you keep

it on the server. Makes it easier to keep it up to date and we don't have to download terabytes with the map of the entire planet if your client just needs to draw the map of a city. Just keep it on the server and let the clients access the part that they are interested about.

However, you still have the problem of delivering a compatible client that can connect to your server. Eventually this problem was solved by building a universal client, which is called a Web browser that can connect to any other application in the cloud. You don't have to change the browser when you change the application that you're using. The software in the client is the same and it's just so flexible that it can work with any other back-end system.

Business model: how to sell

Components vs. Services

- Component developers charge on a per-deployment basis: whenever a new client downloads the component.
- Component upgrades may be sold separately to generate a revenue stream
- Components can be licensed to be redistributed within larger systems and developers can demand royalties from the revenue of the final product
- Service providers can charge on a per-call basis: each time an existing client interacts with a service by exchanging a new message.
- Service providers can charge a monthly/yearly flat access fee
- Services can be made available for free and providers can support them with advertising revenue

If you are trying to make money with software, what are the opportunities to do so if you work with a component-based approach? The idea is that you make money every time someone needs to deploy your software. In the old days your customers would need to pick up the box with the CD and would have to pay to carry it out of the store. Now the equivalent is that your credit card gets charged when you click on the 'Install App' button in the App Store.

Every time this installation happens, every time there is a new deployment, you make money.

We all know that software is never finished, never completely done, especially software that is being actually used. Every time you make a change, every time you improve it, is another opportunity to extract money from your user community. At some point you decide that you need more money, so instead of releasing free updates, you decide that the next one about to be shipped is a major upgrade and they have to pay for it. The toolbar icons and the app logo design have changed so much after all.

Sometimes, after you buy the software, updates are free for a year, but afterwards you

have to pay again to get additional improvements to keep the revenue stream flowing for the developers.

These examples hold if you're doing retail: if you're selling the software to the end users directly.

However, there is a whole other industry of OEM integration, where you are selling components to companies that integrate the components into applications for the end users. In this case the users pay for the applications and then you get a royalty because your component was used inside the application that was sold.

That's another business model. The advantage is that you don't have to worry about interacting with and supporting so many users; you just have one client and then the client will resell your software to the larger audience, maybe in combination with other components.

These are the opportunities to make money which are implied by this simple – but revolutionary – idea that you can actually make people pay to get the right to use a piece of software. It was not always like this, because the software used to come for free together with the hardware. And there was no point in selling software by itself because it wouldn't work without the hardware. So you pay for the hardware and the software is just an afterthought. At some point somebody thought that they could actually make money by selling the software by itself. And with this simple idea, you know the story, one of the largest fortunes in the world was made.

However, at some point the Internet showed up and disrupted this model.

When we turn our component into a service available over the Internet. Other opportunities to make money appear. For example: every time users click, every time they interact with your service running in the cloud, you can charge them for that.

If you think about how often does someone install a piece of software, as opposed to how often they use it: this is a paradigm shift.

Having one exchange of value with one interaction when users download and install your software doesn't provide any feedback on what they are doing with the software. Switching to a business model in which every day every time that they interact with your system you are going to be paid changes everything.

Pay-per-use could be too fine grained. Maybe it's complicated to track. And there is a risk that it becomes too expensive if users use it a lot. It's also possible to select a flat-rate subscription model: to access my system, you have to pay every month. You get a discount if you pay for a year in advance. This is often found with software as a service (SaaS) providers.

More and more software is available for free. So if you make it available for free, how can you make money with it? One answer is the so-called surveillance economy where users are hooked into using free applications and then everything that they do with the applications is tracked, mined and monetized, for example, to sell targeted advertising. If people give you something for free, you should wonder how can they survive financially. And what are they doing with your data? They may be going back to the original model where the software is free, but you pay for the hardware to run it. This will tempt you in using the software as an incentive to drive hardware sales. Like when you need to buy a new phone to get the latest free updates of your favourite apps.



IT as a "manufacturing" industry
(ship software components)

IT as a "service" industry
(publish software on the Web)



In general, while going through the software lifecycle there is a point in which money is exchanged. For example, between the release and the installation, at deployment time, or later at run-time.

While pay-per-deploy is the original model tied to software components, pay-per-use was not possible before because after the software gets installed, users run it on their system, which will be offline. This makes it challenging to track what the user is doing.

We can see here the two reference examples of the two different models for the software industry. One follows the metaphor that we are manufacturing software components. This means that at certain point they come out of the factory. As components get finished, they are shipped as part of the release; and that's when you can sell them. It turned out to be a highly successful concept, especially before the Internet.

Exploiting the opportunities of the Internet, we can make the software available as a service through the web. We still need to build the software, but the software is not something physical that you will ship out. You just deploy it on some servers in the Cloud.

What really matters is: What is the quality of service that you provide to your users?

You switch from a product-oriented type of industry into a service business where you establish long-term relationships with your customers, where customers come together with you to create value, as opposed to just taking a box off the shelf and walking out the store.

Design decisions

Components vs. Services

- Constraint: Encapsulate implementation behind well-defined interfaces, free choice of connector
- Decision: to buy or to make a component?
- Promote reusability and independent component development
- Constraint: Remote connector (e.g., Message bus)
- Services operated by different organizations: high availability, security/trust and loose coupling (independent evolution)
- Many service providers compete to deliver "Software as a Service" from the Cloud

Let's look from a technical perspective at what are the differences between components and services.

Let's just recap the definition of a software component. We draw a boundary between the component in the rest of the system so that we can encapsulate the implementation behind a well defined interface. Components are meant to be connected to each other using a suitable software connector. In this case we don't have a constraint on the type of connector that we can use.

The main decision once we identify that there is a specific component in our system is whether we are going to buy it or to make it ourselves. If we buy it, it means that somebody is selling it to us and we go back to the previous discussion on the business model. If we make it in house, we need to write the code and go through the all the quality assurance processes before we can release it in production.

The purpose of all the component technology that we have seen so far is to make it possible to reuse components. And make it possible for them to be developed independently from each other. You decompose an architecture into different components so you can parallelize the construction work: you don't want to wait for a component to be finished before you start with the next one. You can do all of them together because they have well defined interfaces.

If we switch to services, we assume we can access our software through the Internet. The critical decision becomes: how are we going to connect to the software if it is delivered as a service? We have to go through the Internet therefore we will have a remote or a distributed deployment, as a consequence this puts a limit on which type of connectors we can use.

It doesn't make sense to invoke a service running in the Cloud while attempting to use a share memory buffer to exchange the data. The service is over there, your client is over here, there is no way that you can use local shared memory to send and receive information. Still, there are various connectors that are possible. As an example, the most frequently mentioned when we hear about services, is the message bus, which used to be known as the enterprise service bus (ESB). Due to its asynchronous nature, it also gives you the better availability.

While in the original component paradigm you would need to download and install locally the software, this would imply that somehow - not necessary in a legal sense - you would be owning the piece of software. The license to use it gives you the right to install and start the application on your computer. Nobody else is doing it for you: you are in control of the operational side. You don't necessarily build it yourself, but once you buy it, you download it and you run it so you operate it yourself.

With services, it's exactly the opposite: the software that you use is operated by someone else. This other organization is responsible for making it accessible to you with good enough quality, including availability. Additionally, there are multitenancy issues related to trusting that when you send your data into their system, this data is not, for example, revealed to your competitors. It is theoretically possible to send encrypted data into the cloud so that it can be processed there without decrypting it, but it is super expensive to do so. These are things you have to do if you cannot run locally the software when you send the data to the other side, you trust that the data is forgotten once the response comes back.

The fact that services are operated by someone else means that someone else is going to make decisions on their life cycle: they will switch it on and switch it off whenever they want. And they will apply changes to their services without necessarily warning you. Some popular service APIs post countdown timers on websites announcing breaking changes years in advance so that client developers are supposed to know in advance that the changes are coming. Especially if the changes are incompatible, they need time to prepare for that. In other cases you wake up one morning, you discover that your system is broken, because the service provider went bankrupt and they have disappeared. This is similar if you buy a flight ticket you go to the airport and you discover that the airline is bankrupt and your ticket is not going to take you anywhere.

This can always happen and when you make a choice to rely on a service, your software depends on its availability and you are willing to take the risk it may disappear.

The other interesting aspect is that once you can access the software as a service in the cloud you have a whole market place with a set of competing providers that offer you their software. You have a choice where to go and which software to use at different price points, with different features, delivered with certain qualities and, most important, associated with a brand, which you may or not trust, also depending on the reputation of the provider.

Components were born in an offline world, so component technology is much older than services. Still, you can have remote and distributed components. We still call them components if the organization that is running them is the organization that bought them. So you buy the component, you run it yourself even if you deploy it across a distributed environment. You are in control of its life cycle.

A service is a component that is remote but is also provided to you by a separate organization that is in full control of its lifecycle. All you can do is call the service and hope that it answers you. There can be some expectations and guarantees, but you cannot do anything if the service provider is offline. Or when a service disappears.

With the component you still have the bits of the software which can run as long as you can deploy them in a compatible environment.

In other words, if you are in a company and you have the IT department that is supporting the applications within the company, when the lights go out, if the application stops working you have a phone number that you can call and there is somebody from the same company that answers it. This is internal support for ensuring the availability of the component. If the component is delivered as a service, the number that you call will be answered by an external entity, and you hope that they still answer and fix the problem with a comparable speed.

Overall, the main difference between remote components and services is not really a technical one: it's really about ownership and responsibility: who to blame for unavailability.

Technology: how to use

Components vs. Services

- Component need to be packaged to be deployed as part of some larger application system
- Components need to be downloaded and installed, assuming they are compatible with the existing framework used to develop the system
- Problem: There are (too) many incompatible component frameworks
- Services need to be published on the Web by advertising their location to potential clients
- Services are invoked using standard protocols based on HTTP, XML or JSON
- Problem: Services are distributed, remote components outside your control. **Beware if they become unavailable!**

Let's look closer to more detailed technological steps to deliver the software so that it can be integrated within our architectures. Delivering software as a service from the cloud is different than just releasing your software, publishing it on a website so that users can download and install it and run it locally.

We've seen already the build pipeline that we use to go from the original source of the software into the component that we can release, and we can install. There is a point in which you are transferring bits and you copy them on some environment that you control, and assuming that you are downloading multiple components and you want to use them together in the same architecture, you make an assumption that the components are compatible.

Component interfaces must be compatible with each other, but also they need to be compatible with the underlying framework they require, with the programming language runtime. In other words, if you take a random example, if you go to the Eclipse marketplace and you look for plugins, you look for components meant to be deployed on this platform, you expect that you will find components written in Java. The chances that you can easily take a component that is written in Ruby – they are called gems – which are meant to work within the Rails framework, and deploy it within Eclipse are slim.

Those are completely different technological platforms. NPM modules cannot be installed using “apt-get”. Package managers work great as long as you stay within the same platform. If you try to mix components from different languages and different frameworks then you will have some problems due to the large number of programming languages and even larger number of different, incompatible by design, frameworks.

Some of you might mention the word "Docker". This is a recent development which is trying to solve such component incompatibility problem and provide standard way of packaging and installing the component no matter programming language it is written in as long as it can run on a Linux-flavoured runtime. Components deployed in containers expose programming language independent interfaces which make it easier to connect them together. Such standard interfaces were originally invented to connect services.

If we switch to services, the goal is completely different. One of the reasons why there are so many frameworks is that vendors make money with the frameworks as they attempt to establish a virtuous cycle where you have a successful framework because there are many developers that write components for it, and the more components are there, the more developers will use the framework to find components for their applications. And therefore, if you establish a new framework you want to lock in the developers and – even if the programming language appears to be the same – you will define framework interfaces and APIs that are incompatible. Once you write a component for that framework, it will be very challenging to port it somewhere else due to its framework-specific dependencies.

This makes sense if your business model is tied to getting people to use your technology and make them dependent on it. If your goal is to make your software accessible through the Internet, then you want the software to be as compatible as possible. You try to make it accessible from clients that are written in any programming language. You seek the freedom to implement your service behind a standard interface running in the cloud so that you can implement it with the most suitable language.

You also don't want this information to be actually visible or known to your clients. It wouldn't make sense to state: you can use Google, as long as if you use this particular version of C++ with this compiler to invoke its search API. If you switch the compiler, then you risk becoming unable to call this particular Web API.

This has been a huge evolutionary shift in mindset and also concerning the technology base. After decades spent into standardizing all the necessary protocols so that you can exchange data, you can represent it using a programming language independent way, today most interfaces use HTTP as a way to transfer the data and then on top of it they use either XML or JSON to represent the payloads, the message content sent back and forth.

Another important aspect is the discovery of matching interfaces. We already discussed about registries, about directories. They are also relevant with services because you need to find out where on the Web a reputable service provider that you can trust can be found.

For these reasons we have a big difference, services put emphasis on interoperability, so that the service interface is accessible from as many clients as possible. Components face the challenge of portability: components written in some languages need to run in different, potentially incompatible operating environments. This is not a problem with services, because the service provider is in control of the operation environment. They can optimize it and choose the most suitable one to ensure good performance, scalability, and availability.

Components vs. services also impact software piracy. If you want to steal a copy of the software, if your business model depends on people downloading and installing the software, then it's very easy to make a copy. If you are delivering the software as a service, nobody will ever see your software. The software stays safe locked inside your data center and people just call it from all over the Internet. To run a google search you do not have to install your personal copy of google.exe – such piece of software is never going to be downloaded anywhere. It stays within the boundaries of the service provider, which however will need to deal with account sharing and fake API keys.

While services can be made accessible from anywhere, the remaining issue is that they run in a distributed environment outside of your control. Once you draw a dependency in your architectural model to represent: our system depends on a service API, there should be a big red flag going up: What if this service API becomes unavailable? Such unavailability could be temporary – the service becomes unreachable for just a few minutes and nobody notices. Or the service disappears for a few days and then you read about the outage on the newspapers. Or maybe the service provider goes bankrupt and then your system depends on something that doesn't exist anymore.

If you deliver a service which depends on external services, their availability will have a strong impact on your own availability. How can you guarantee your own system will actually work if you are not sure whether your dependencies will be there when you need them?

Availability



When software runs in the cloud, sometimes we have thunderstorms. The system is either not reachable because there are network problems – what if you try to access a system while you’re traveling around with your mobile device? What if you are crossing the Swiss Alps and you don’t have very good connectivity? Will the software work in offline mode? You have experienced a network partition. The user device is disconnected. How is this going to affect the user experience? Are you going to just disable the user interface and stop the application from working? Once you reconnect you enable the user interface and everything works again because the backend in the cloud is reachable. How can you deal with this problem? Add a backup link. Make the connectivity more reliable. Switch from the WiFi to the 4G or 5G antenna (or viceversa).

We can generalize this concept: this is a fundamental idea that we can use to improve availability is to introduce redundancy into the architecture.

If something stops working, you have a backup. You have an alternative that hopefully will take over so that the system still works.

We have seen that redundancy and replication also help with scalability. For example, consider a content delivery network, we need to serve read-only assets and we can spread them out throughout the CDN so that we can improve the bandwidth and latency towards a large number of clients.

If we make multiple copies of the data, we have introduced redundancy: somewhere we have a mirror which we can use if the main site is offline or unreachable.

But is it a good idea to stop the user from working when offline? If there is a glitch with the Internet, should the world stop? Within every architecture there should be the possibility to revert back to a state before the Internet existed. Once upon a time, we used to know how to run software without the Internet. We should still be able to do it

today. Just download a local copy of the software and the relevant data. Even if we are disconnected, we can still allow users to work with it. Then at some point we assume that the connection comes back and we can synchronize: apply the local changes to the server-side state, while fetching any updates from the server. There is a price to pay: conflicts need to be detected and possibly resolved.

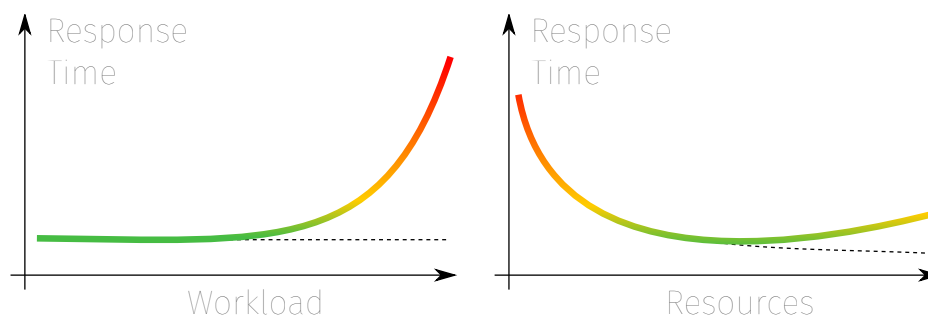
Availability Questions

Is it running?

Where is it running?

Is it reachable?

How long does it take to reply?



How long are you willing to wait for the reply?

The first question you can ask to measure whether something is available concerns whether it is running or not.

What is the current state of execution of your system? The system started, it's running, it's available, it's ready for the users, or not: the system stopped, crashed, it's not running, it is running through an infinite loop, it takes forever to respond, it's not available.

Now if it is running, you also may need to know where it is running. That's a different level of awareness: from the user perspective, it is enough to know: "if I need to use it, it's running and I can use it". But from an operational perspective, you also should have a more precise idea: we started it and is now running on this machine. We deployed it using this container and while we run the container on this machine, if something happens to that particular machine, this component will be affected.

If you just know that it is running somewhere, you have no ability to predict what kind of failures will impact, whether it keeps running or not. If you know where it is running, then you know which machines you should protect and you should be careful not to unplug them at the wrong time. This is a most important concern from the service provider side: I'm running it, it's over there and I'm watching that nothing bad happens to that machine.

Since it's remote, you also have a concern from the client perspective. Even if you say it's running, you can reach it, but it doesn't work for me. So availability may be

affected by network reachability issues. If you get disconnected, having a second network connection can help mitigate the problem. This may help to improve the reachability of the service.

The fact that it is running somewhere is necessary, but it's not sufficient if you cannot reach it, so both of these conditions have to be true: It has to be running and it has to be reachable. Then we can start to look into the details and we can ask: how long does it take to reply?

I'm going to upload my video on Panopto. And it takes one hour and a half to process it. Is that really available? Yes, it's reachable, I can transfer the data and it's answering when I refresh, but there is this progress bar that is going up and down. You see something that tells you we are 95%. And then after one minute it's gone back down to 20%. So it's particularly hard to estimate when you're going to get the final answer.

One way to explain this is to look again at the scalability curves: we can see the response time depends on the workload: how busy the system is. Green values show that the system has an acceptable response time. How long does it take to reply? Not too long, if I can get the video uploaded before the lecture starts, it's fine. But if we are in the red zone, we still get a reply, eventually, but the reply is late. If the reply is late you have to ask yourself, are you willing to wait that long for the reply? If the reply comes too late, it may be correct, but it may be completely useless.

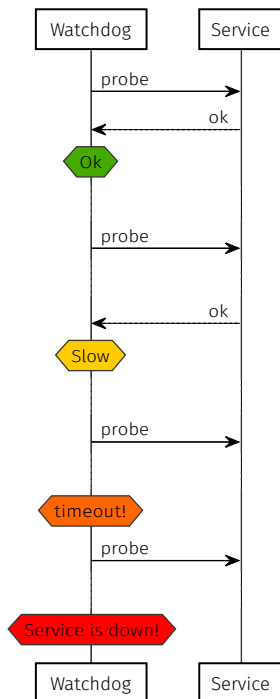
When you worry about the availability of a system. Only as a first approximation: It can be a black or white. If it's running and reachable, it's available. When is down and offline, it is not available.

Then you can start to wonder, do I really care about availability unless I need to use it? And when I use it, when I need to use it, is it going to give me a timely reply? This way we can translate the availability question to ask: how long does it take and how long am I willing to wait for it? This can also be seen as: How soon will the client timeout? When you send a message, you may expect a response; but if the response doesn't come within a reasonable amount of time, you can conclude the response will never come. You're simply not going to wait anymore. Even if the response comes later, you no longer wait for it, so you are going to miss it, because you already concluded that the service is not available.

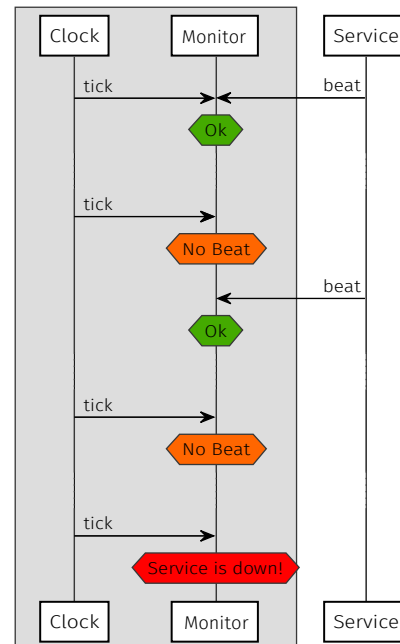
From your perspective, the perspective of the observer, or the client: if a system is too slow, it is not available.

Monitoring Availability

Watchdog



Heartbeat



You cannot know if a service is available unless you try to interact with it. To keep track of the availability, however, we don't want to rely on the users reporting outages. Instead, we want to have an automated system that can continuously check whether the availability state of a service.

Given the need of an observer to determine the availability of a system, there are two different options for monitoring the availability of services, depending on whether the service is passive or active.

One is by periodically sending probes from another system, called the watchdog. The watchdog periodically probes the service whose availability we want to check. In case the service is active, we use a heartbeat message which is periodically sent from the service back to the monitor (or passive watchdog).

The watchdog is a component whose purpose is to call the service and check whether the service gives an answer within the expected time. The watchdog (active) calls the service (passive). If it receives a timely answer, we can conclude at this point in time the service is available. If we send another request, another probe, after some time, maybe it takes a bit longer to answer. The watchdog can not only measure whether we get an answer or not. That's black and white, but it can measure how long does it take to get an answer. Based on the response time, we can determine whether the system is fast enough. However, if we send a probe and we don't get an answer. We cannot wait forever: there will be a point in time in which we decide we have waited long enough. The watchdog times out. And then we try again. If we don't get an answer after a certain number of attempts, then we can conclude that there is a availability problem with this particular service. We go from a state of availability where we get a timely answer, to an intermediate state in which the performance is degrading. Eventually we lost the service because we have been patiently waiting for a very long time but the service still doesn't answer. So that's the worst case scenario: a slow failure. It's also possible that when we

try to connect to the service, we immediately get an error. This will be a faster way to determine that the service is not available.

With this type of watchdog, the service is passive. So the service is just servicing yet another request. It doesn't care or it should not need to know whether requests are coming from a normal client or from the watchdog. The watchdog is just one more client that will check whether the service is capable of processing some probe request. Still, you need to decide whether the watchdog calls a normal operation of the service, or if you have a special interface that you just use from the watchdog to check whether the whole service is available. If you have a dedicated and separate probing endpoint, you need to deal with false positives and true negatives, when the probe times out but the main interface is still working. The same holds if you use separate network planes to transfer production traffic independently of the systems management messages.

If our goal is to monitor the availability of the service. Is this solution enough? What could possibly go wrong?

We're making an assumption here. We have two components. One is watching the other. What happens to the watchdog itself? Who's watching the watcher? Consider setting up a cross configuration, with a ring or a chain of watch dogs that are checking each other if they're available. And then they also collectively check the service. We need not only to worry about the availability of the service, but also of the infrastructure that is checking whether the service is available or not.

The service is available if it answers to the probe within the expected time. The probe can be any message accepted by the service API. The challenge of having a watchdog that probes the service is to make sure that if the answer is successful, this is a representative observation where the same behavior would also be perceived by any other client. Better avoid the situation in which the watchdog thinks that the service is available, but then the service will not work for any other client.

If you don't get answers, you insist, you keep sending multiple probes. Only if no answers comes back, after multiple attempts, you can conclude that the service was not available. How many times should you keep trying?

It's also possible that when you try to connect, you cannot reach the servers. Also in this case, from your perspective there is an availability issue. But the problem could be due to networking issues on your side and although the service is running, you cannot reach it. This is why you may need multiple watchdogs to reach the conclusion: the service appears not to be available from everyone. Is it down for you too?

There are some scenarios in which you can afford to have a service which is active. This means that the service will be sending periodically heartbeat messages back to a monitoring component, which will basically check whether the messages arrive when they are expected or not. This alternative design is also used in practice with the assumption that the service is actually going to be notifying the watchdog about its existence, for example when registering its location with a directory.

We have a service that periodically notifies the passive watchdog of its availability. That is, the watchdog expects to hear from the service every so often. In case this doesn't happen, then basically the service has skipped a heartbeat, or the watchdog missed it. Probably something happened on the other side. There will be a window in which the watchdog is listening for messages; if the message arrives on time, it can conclude that the service was running. If you don't get any message you can start to worry, but the message could still arrive late. We have a slow service that for some reason is behind the schedule.

If however this situation persists like in the example, with two missing messages – it could be more, depending on how sensitive is the watchdog – we could conclude that the service was lost. While the monitor can only show when was the last time we heard

from the service and how many heartbeat went missing, if this time grows above a certain threshold, the watchdog may start to actively probe the service to confirm that indeed it is no longer running.

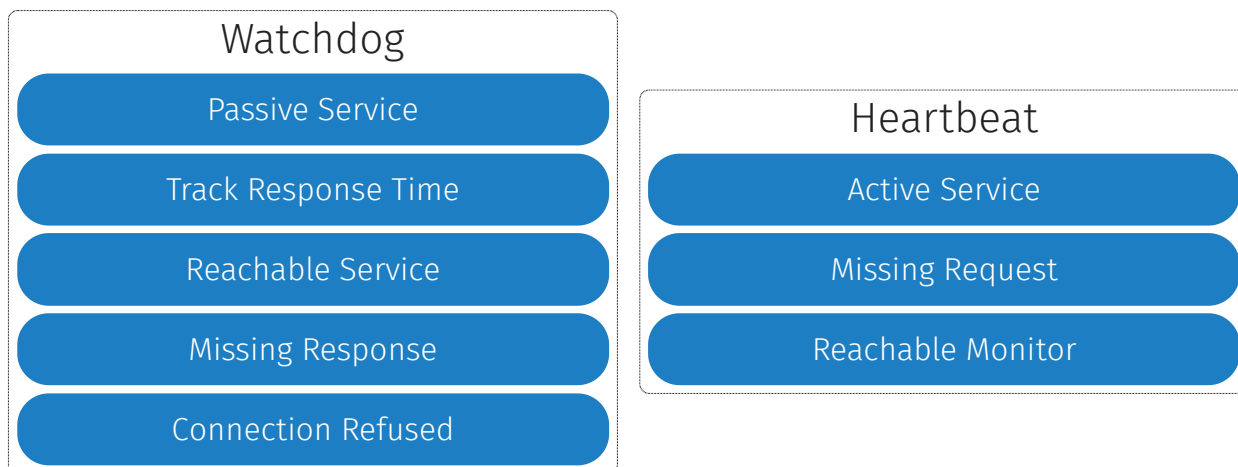
In the same way the directory helps to discover the location the other components, also the watchdog helps to keep track of the availability state of the other components. Both suffer from the same limitation, since the watchdog cannot monitor itself and the directory location cannot be looked up on the directory itself.

What is the role of the clock? The clock tells the monitor when to expect a new heartbeat message. There has to be an agreement between the clock and the service regarding how often the message is sent. This is similar to the decision on how often the watchdog should probe the service. You don't want to do it too often, because otherwise the watchdog is overloading the service. But the less frequently you do this, the longer the delay will be between when the service goes down and when you can actually detect it.

In the same way that you have to decide for how long or how many attempts are you willing to wait before you give up and declare a connection timeout, we also need to configure how frequently to perform each attempt. It may also be a good idea to employ exponential back off in case the probe itself may overload the service.

Another aspect to consider, for example with deployments in an Internet of Things environment, is the energy consumption induced by the watchdog or monitoring system. Sending probes and heartbeat messages has an energy cost. The more you send, the faster the battery gets depleted. So you need to trade off the delay with which you detect outages against the duration of the window in which a battery powered watchdog can run unattended.

Which kind of monitor?



To understand the difference between these two different solutions, I would like to ask you if you can fill out the different properties for each of them.

Unlike a monitor, with a watchdog you can monitor not only whether the service is available or not, but you can also track its response time.

If you use a watchdog, the service has to be reachable, since the interaction starts from the watchdog which has to be able to contact the service.

You can use an active watchdog when the service is passive: the service by itself is not aware that is getting probed. It does not have to do anything extra so that you can track its availability. The service is just deployed and started, and then the watchdog calls it and the service replies as if it would receive requests from any other client.

The watchdog is going to periodically send messages to the service. This is the difference with the heartbeat, in which the service has to send the messages to the watchdog. And this means that the service, in addition to all the other things that is doing, has to do one extra thing, it has to remember to send the heartbeat on time. For this to work, the monitor has to be reachable.

When it comes to the errors, then you should put yourself from the perspective of the component that is monitoring the system, so it's the perspective of the watchdog or the monitor. If you send a probe and after a while you don't get a response, then you know that the service is probably not available. If you send another probe and the response is also missing, then you can raise an issue with the availability. If you try to send a probe and the connection is refused. That's even a stronger sign. You don't need to wait to detect that there is a problem, since the service does not accept your connection, you know that it will be impossible to send out the probe. As opposed to waiting for a missing response after sending the probe – the probe goes out into outer space and you listen into the void for a faint echo – if when you try to send the probe, you cannot even do that, then it's clear the service is not available.

From the hardware perspective the heartbeat will help to detect availability issues when there is no request coming from the service that is supposed to send a heartbeat messages. You can still get a connection refused or a missing response, but since the service is active, these errors will tell the service that monitor that is supposed to receive the heartbeat is not available. From the monitoring system perspective this is not useful information.

If you introduce a watchdog, you have a service that is just there waiting for clients and the watchdog becomes yet another client that has of course to be able to reach the service. It has to be able to connect and send the request. If the response comes back, you know that the service is available. If you choose the heartbeat alternative, you are in the opposite situation: you have a service that is active and needs to be able to send messages to the monitor. If the monitor doesn't get the request from the heartbeat message from the service, it knows that the service has disappeared.

In terms of connectors, since there is no response required for heartbeat messages, they can be sent asynchronously over a message bus, while the watchdog uses a call interaction with a request followed by a response message.

Availability Incidents

No Impact: The service is down while no-one needs it. A client may retry the call and find the service back up again (assuming idempotent interactions). Automated failover mechanisms can mitigate the failure.

Low Impact: Manual intervention required (few hours) to recover the service, some customers start to complain. Easy to spot the cause of the failure, more difficult to find the operator/developer in charge to fix it.

High Impact: Media visibility of the (1+day-long) outage. A whole disaster recovery team needed to bring it back up.

Let's see what happens when something is not available. Of course, if you are down when nobody calls, nobody notices. There is no impact. Just like software nobody uses has no bugs, services nobody calls are always available.

What happens when things go wrong? Sometimes you might have experienced a temporary glitch. You just retry the call and it works again. Something failed, but there is some automated recovery process that takes over and fixes the issue. The impact is very limited, the failure has been contained. Maybe only one user notices something, but the other users are not really affected.

We can also have incidents with low impact. There is no automated recovery. We need to have some manual intervention. Typically the delay is caused by the fact that we have to find the right people who know which is the right button to click on and have the right to restart the system. It's easy to fix it once you find the right person that knows how to do it.

When the down time grows beyond a few hours, many people start to notice and you start to hear complaints. Since, however, the system works again after you restart it, user disruption is contained.

You can also have a major impact when recovering from the disaster requires to set up a whole team and execute a small project that takes days to succeed. Imagine a popular service, some kind of social network, when such incident happens there will be some visibility in the news. The media will report the outage. Or the outage will be reported by users through their social network accounts.

In the same way you need multiple watchdogs to monitor one another, we need multiple social networks because people can vent on one social network and complain about not being able to use the other one and vice versa.

If you subscribe to the view that there is no bad publicity, as long as people are talking about you; you can also take advantage of the spotlight to enhance your brand visibility, be creative with the error messages (the big failure whale comes to mind). Even though people cannot achieve what they want to do when they try to use your service that is not available, they still get a little bit of entertainment and they don't go away totally frustrated.

Downtime Impact

Revenue Loss: While the service is not available, the business is disrupted and the revenue may stop flowing

Reputation: Customers are unlikely to trust a service with their traffic and their data if the service is not responsive and the data is out of reach when they need it. Planning and announcing outages in advance as well as some degree of transparency in the recovery process of unexpected downtimes may help to manage customer expectations.

Recovery Cost: The activities to diagnose and repair faults may require a significant effort and to provision backup resources

Refund: Long lasting outages may violate Service Level Agreements and trigger refunds (in cash or credit for future usage) to avoid litigation

What's the impact of worst-case scenarios? You lose money. When something doesn't work, the business stops, economic value creation stops, financial flows stop. For example, there is a particular system to get payments from your customers. If this doesn't work, you're not getting the money. This can quickly become a critical issue, especially for high throughput money making machines, every second downtime can translate to millions of lost revenue.

There is also an impact on your reputation. Clients over time learn to expect a certain quality of service. If when they come to you, you don't deliver on the expectations you created. If you break your promises: your service is not responsive, users entrusted your service with their data and oops, your irreplaceable bits have been lost, we are sorry for the inconvenience. This is a major issue. It takes a long time to build up a reputation and a very short time to tarnish it.

Expectations matter. There is a big difference between planned outages, which can be announced beforehand and the ones that happen without any previous warning. If you promise that you will be available again, after a short down time, then as long as clients are informed, they will not be so disappointed.

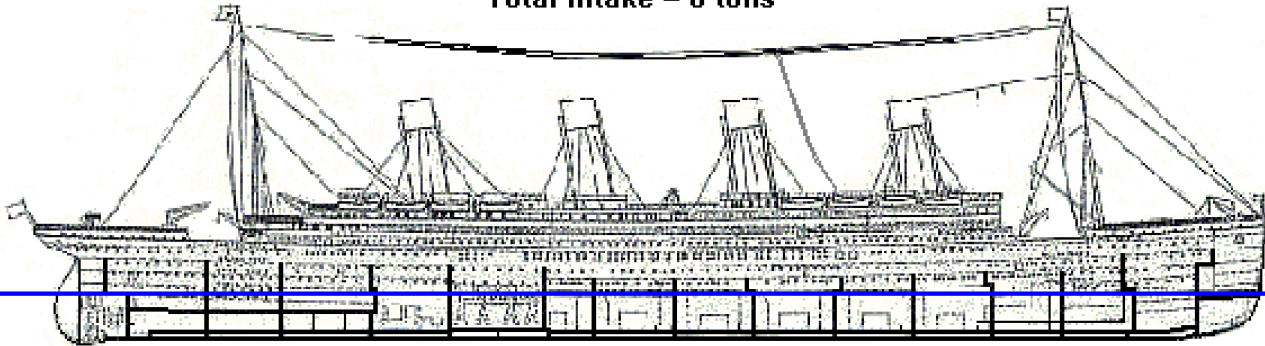
In 2020, this can also happen in the opposite way. There is now an expectation that when you try to order something in an online shop, typically the delivery windows are never available because they are so overloaded at the moment. One day I was surprised to find an open delivery window right away. I connected and everything was green. I placed an order and they actually took my order. Just like in the good old days. You need something and they are available. They will take care of it. And then a few hours later, I received one of those emails: actually there was an error, our service was mistakenly available at that time. It was supposed to be closed, but for some reason it was misconfigured and it was open and taking the orders. So we are going to cancel the order and

refund the payment. Thank you for the understanding.

Availability is affected by overload and lack of capacity to scale beyond a certain workload, and thus it can be improved by investing in additional resources (assuming the architecture can scale to benefit from them). Restoring availability after failures, requires to pay the cost of recovery. How much effort do you need to bring it back up? So is it just a matter of restarting something? Or do you have to rebuild an entire data center somewhere else because the hurricane flooded the current one? How much time do you need to get the backup resources online?

Depending on your service business model, if you have some subscribers that are not able to use your service during a certain time because of the outage, you might start to get some liability. Not only you lose revenue from pay-per-usage clients, but you also have to refund subscriptions or give them credit for future usage because your system was not available.

Undamaged Condition
Total Intake = 0 tons



titanicology.com

Contain Failure Impact

In general, with services, the expectation is that you offer them with 24/7 availability. Users from all over the world should be able to access and use your system no matter what, no matter when or where they are. And this is a very challenging thing to achieve.

Let's see what kind of architectural patterns we can introduce to be able to actually do that. How can we contain the impact of failures? As you can see in this model, there was this idea that even though an iceberg may punch a hole through the hull of the ship, only one compartment would get flooded. The ship will not sink. How many flooded compartments can the ship survive?

The idea is that you try to isolate the impact of the failure, you try to protect the overall system, even though parts of it might suffer. You need to prevent cascading failures to bring down the whole system. You install fireproof doors which should remain closed. You put travelers in quarantine to stop the plague from spreading. You dig a second side-tunnel across the mountain. To access the Internet: wireless as well as wired access, thank you, even for laptops.

What's the simplest idea that we can introduce in our software architecture to be able to do this? It boils down to the notion of redundancy. You want to have redundancy in your architecture if you want to increase the availability and reliability of your system. But with redundancy, the software becomes more expensive to build and to operate. You can ask your accountant who is skeptical about the cost of the redundancy if you can afford the cost of the recovery.

Retry

How to recover from temporary failures?

Retry the failed operation

If the call fails or no response is received (a timeout occurs), repeat it.
If a message is lost, resend it.

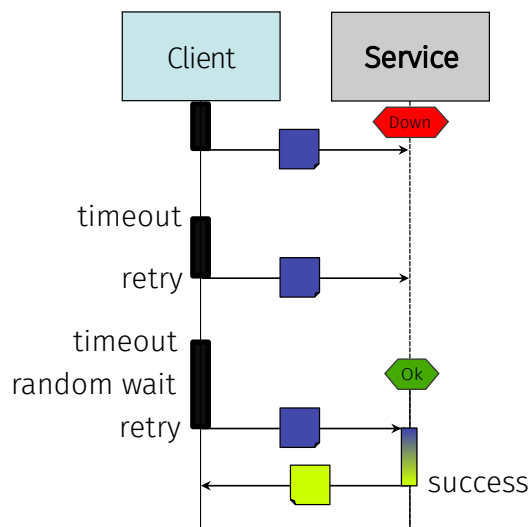
Let's start with a very simple form of redundancy over time. If you can expect the failure to be temporary. From the client side, when you try to invoke a service and it doesn't work, what can you do? You just try again. Try again later, hoping for the best. You don't give up, keep trying until it works.

There are many events which may lead to the opportunity to retry a synchronous call. The call doesn't go through, the call might fail, or maybe a timeout or a disconnection occurs during the call. You are sure the request was sent, but you don't get a response. After waiting long enough, just hit the refresh button, maybe it will work now.

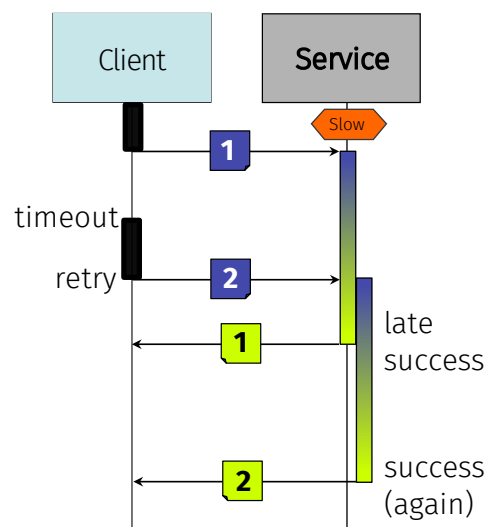
If something goes wrong while sending a message, just resend it. This implies that you need to keep a copy of the message until you're sure that the other acknowledges its receipt. Only when the other side confirms that they received the message, you can forget it.

Doing this will improve the chances that the message gets across, eventually.

Retry



Retries eventually succeed
(with temporary failures)



Retries introduce duplicates
(redundant rework)

How does the retry mechanism work? We try to talk with the service. But the service is down, so there is no answer. Even if we wait for the answer, the answer doesn't come back. At some point, we have a timeout that will trigger the repetition of the same call, the same request will be resent. We're lucky, the service is back up. Now the service is available and we get the reply back.

To retry is to be optimistic. It assumes that the service will eventually recover, that the failure is temporary. It also assumes that the messages can be resent as many times as you want. What kind of messages can be repeated? Can you always do that with arbitrary messages with arbitrary operations over service interfaces? There is a problem.

When we repeat a message, when we retry, we introduce redundancy in the system and this can be a problem. Let's take a look at this other case, in which we have a slow service. Even if the service is slow, the first message still gets across. The message gets processed, but it takes longer than the client expects. If the client is not patient, if the client resends the message too soon, we have now a copy of the message that will be processed. The client will actually receive not only the response to the original message, but it will also receive the response for the second one.

Since we have introduced a duplicate message into our system, an operation that was completed slowly once is actually done slowly twice. As a result, we have more load on the service due to the client timeout being too short. The client was not willing to wait anymore, so the service had to repeat the work twice.

Retry

- Which operations can be safely retried?
 - **Idempotent** operations (result does not depend on how many times they are executed)
 - **Stateless** components (result only depends on input, no side-effects on the component state)
- How soon?
 - Retry **immediately** (risk of overload by refresh)
 - Retry **after delay** (exponential backoff)
- Which is better?
 - fail after the first try (soon)
 - keep retrying until it works (maybe forever)

Think about what kind of messages can be retried? Can you give me an example of a message which is not possible to safely process more than once? What could go wrong if we process the message twice?

Maybe the request is about reading some information. If you retry, you may get 2 copies of what you were trying to read. Maybe one copy is more recent than the other. But there is no side effect on the service. Read operations can be retried.

What if, for example, we have a financial transaction. The semantics of the messages is about to deposit some money into the users account. Of course the user is happy if you do it twice. But the bank is not so happy. However, if you attempt to withdraw money. You want to make sure that you withdraw the money exactly once.

Any kind of interaction which changes the state of the service incrementally, based on the previous state of the service, cannot be retried. Otherwise, side effects accumulate and both the result and the state of the service will be different depending on whether the request was retried or not.

The idea is that retries should enhance the availability but not change the basic functionality.

If the service interface offers operations in which the result does not depend on how many times you do it, they are called idempotent operations, then retries are fine. You can also have stateless components – as we've seen the result of stateless operations depends only on the input. Therefore there is no side effect. Therefore it's possible to just retry the computation. You will waste CPU cycles, but there is no problem with the consistency of the system.

How can you turn every every other type of operation into something that can be retried? You have to put unique identifiers on the original messages and then you have to do message deduplication. You have to remember that you've already seen a message and you shouldn't process it twice. There is a cost to check every incoming message against

the identifiers of all messages that have been previously received. The duration of the time window used to remember previous message identifiers should be set according to the retry timeout.

If the retry doesn't work, we can keep retrying for a while. Maybe wait longer and longer between retries, to avoid flooding your service with too many of these retries. When a service is overloaded – it is not yet down but just on the brink of collapse – a few of these extra retries from impatient clients may be enough to bring it down for good.

This happens more often than you think. Most users have been trained: If it takes too long, just refresh. If the service is slow, it takes too long, what to do? Click refresh and therefore the service gets even slower, even more loaded with even more requests. This can start one of these positive feedback loops in which the longer it takes, the more people refresh and then it takes even longer. It's hard to recover from these situations unless you're willing to stop retrying, at least for a while.

Since we can protect the service from duplicate messages induced by retries, should we always retry? Should we always keep retrying forever? The service is bound to come back up, right? What if we recover the service on a separate server? What if the client keeps sending messages to the previous address? When is the client going to give up and consider doing a directory lookup to discover that the service has moved? And yes, it was already available at the new location?

We should not retry forever, since in the worst case failures are never temporary. The question is: How soon do we give up? When can we escape the retry loop to attempt a higher level recovery strategy? What if we skip the retry at the first sign of failure so that we can report the error as soon as possible?

Retry is a good strategy if you have temporary failures that can be expected to be recovered soon, so called glitches. While stuck in a retry loop, the system will not necessarily report there is a problem, because it's hoping that it will work, at the next retry. This means that users may perceive the system as hanging, and their usual recovery strategy (click refresh) is already being attempted by the system, so it's no use for them to add another manual retry on top of the automatic ones.

The message is that if infinite loops are the enemy of developers, infinite retry loops are the enemy of operators dealing with outages. It would be a good idea to be able to exit the loop and just fail, as opposed to waiting forever while delaying the inevitable. Somewhere there should be a configuration option called: maximum number of retries.

Circuit Breaker

How to avoid retrying forever synchronous calls?

turn a slow failure into a fast failure

If the failure/timeout occurs during a remote call the circuit breaker trips and avoids performing future calls to the failed service

Wouldn't it be nice to be able to remember from the past whether you were trying to interact with a service that was not available? You learned the hard way it's down because you kept retrying for two days, but it still didn't work. So don't even try to enter the retry loop the next time. If it didn't work before, what's the use of trying so hard only to give up after reaching the maximum number of allowed retry attempts?

While it's clear that skipping retries altogether is for pessimists, there is some pragmatism in trying to learn from previous experience.

When we introduce a circuit breaker into the architecture, we specialize the remote procedure call connector to use a smart retry strategy.

The problem of remote procedure calls is that they block the client until the call has completed. While this may not be a problem if the service is available and the call completes with the usual speed. In case retries come into play, completing the call will start to take much much longer than usual.

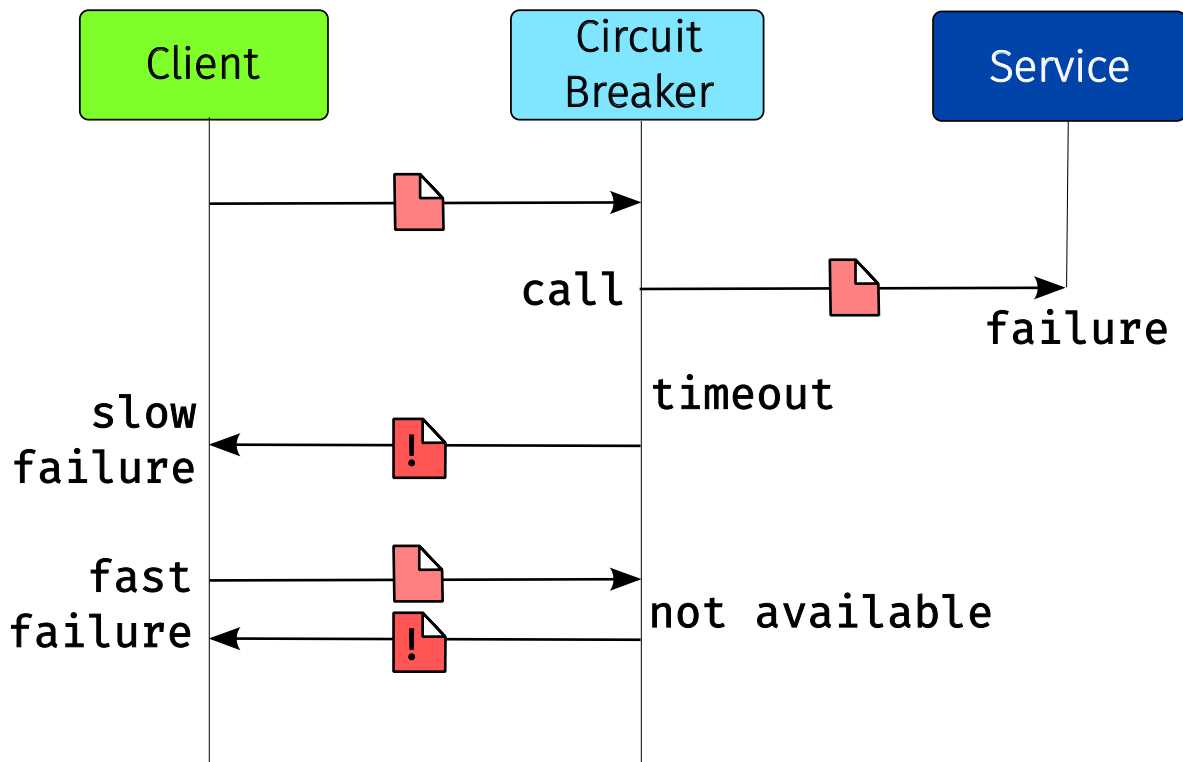
The circuit breaker is a special kind of connector, a variation over the remote procedure call connector which deals with these temporary failures, these availability issues and helps clients to avoid getting stuck into an endless retry loop while attempting to call a service which is already known to be unavailable.

In other words, we want to remember what is the state of availability, or at least an estimate over the last known state availability of the service that we're trying to call. If we expect the service not to be available, we give up immediately. While if we expect the service to be available, we call it. If our expectation is invalid, we switch between the two states.

If it becomes possible to immediately respond that there is a problem, it may be possible to take some alternative action or recover from the client or simply inform the user right away and let them decide if they want to refresh.

For example, if the call didn't work, you can display the cached results from the previous call. You can make users aware of the unavailability without having to slow down their system.

Circuit Breaker



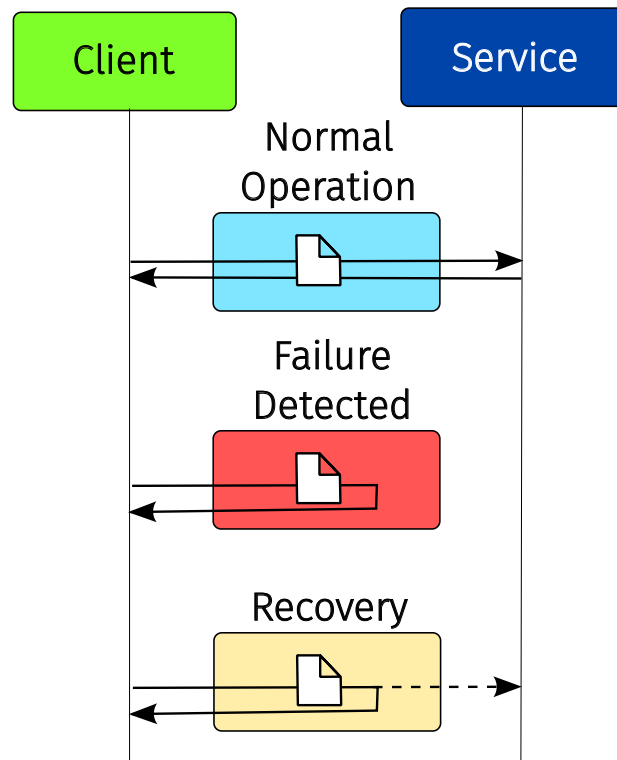
Let's see how it works. If you make a call and the call fails or it times out after some re-tries, then you will stop calling the service in the future. While the first time it takes some time, you have to wait for the timeout after all, the next time the call fails immediately.

We can see here how the interaction works. We have the client making the call. The circuit breaker is in the middle, so this can be abstracted as a connector which takes the call and forwards it to the service. If the service is not available, it takes time before we detect that there is no answer. When there is a time out, we have an error, which gets forwarded to the original client: This service you are trying to call is not available, despite several attempts, we didn't receive an answer.

Additionally, the circuit breaker remembers this outcome. The next time we try to make a call to that service, the circuit breaker will immediately respond with the error.

This may seem like a minor optimization. However, imagine a complex architecture in which you have not only a client calling a service, but you have a client calling service which calls another service which calls another service. If there are a lot of nested calls, a slow failure with one call will affect all the other calls. And it may take a very long time before the client notices that somewhere down the stack a service is unavailable, after everyone in between finally gives up retrying after going into timeouts several times.

Circuit Breaker



Within the circuit breaker, the decision on whether to forward calls or stop them depends on the state of the breaker, which reflects the latest known state of availability of the service protected by the breaker. A closed circuit breaker lets the call through when the service is assumed to be available. When the client makes the call, the service breaker forwards it to the service and the answer is returned within the usual time, depending on the performance and the workload of the service.

However, if the circuit breaker detects that there is a problem with the service, it will switch to the open state in which the next call will be bounced back to the client immediately. The detection can happen by observing the failed outcome of previous calls, or with the help of a watchdog.

The metaphor works like in electrical circuits. When accidents like short circuits happen, the circuit breaker can save your life because it disconnects and removes the power automatically. To restore the original state from the disconnected state, you have to reset the breaker manually, that is after solving the problem which tripped the breaker in the first place.

Tracking the state of the breakers can help to monitor the availability of the corresponding services. One can easily build a monitoring dashboard showing the state of all circuit breakers in the architecture. You notice the ones that have tripped and after recovering the corresponding services, you can manually reset them to let the calls through again. Or you can also try to do it automatically: every once in a while when you get a call the breaker will try to forward it to the service to see if the service is back up. And in case this particular call works, then the breaker goes back to the normal operation state. You start to recover it after a certain amount of time, to give enough breathing room to the people trying to recover the service so that you don't immediately flood it with traffic.

Circuit Breaker

- Isolate the impact of failures (errors and timeouts)
 - Need to determine how many failures can be tolerated before tripping the breaker
 - Recovery: the circuit breaker needs to be reset (manually or automatically)
- Apply whenever you do not trust your dependencies to be available at all times
- The client needs to expect failures and tolerate them (use default or cached response value, temporarily disable functionality)

The main idea of the circuit breaker is that for a synchronous interaction, the client is calling the service and the service needs to be there to take the call. The success of this interaction is highly sensitive to the availability of the service. If you make a call and the service is not available, you typically end up waiting before you give up and conclude the answer is never going to arrive.

The idea is that we can close the circuit breaker and immediately fail the call: “sorry, you’re trying to call something that is not going to answer you“. Clients want to know this as soon as possible. What can clients do when they get such a response? This response is not coming from the service because the service is not available. If clients are trying to read information from the service, this can be cached. If the service is not available with the latest updates, a previous version can be fetched from the cache, whose availability should be independent from the one of the service. Clients will not get the latest information, but at least get a copy from the past and maybe that’s good enough.

If clients are trying to write into the service, the circuit breaker quickly informs them that the update didn’t go through. They will need to retry it again later when the service recovered. So it is not the responsibility of the circuit breaker to buffer messages and automatically resend them. The circuit breaker is not a message queue. Depending on the client, the rejected updated will need to be stored locally, or an error is reported to the users, who can wait for better times to retry the operation.

One important decision regards: how sensitive is the breaker? Is it going to trip on the first failure? Or how many failures should happen before it switches to the closed state? Also, how do you recover it? How do you reset the state? Not only you have to recover the service, after the service is back up, remember to reset its breaker, or clients will not be able to call it.

We introduce this type of solution when we do not trust our dependencies. In case dependencies are expected to go up and down, and we want our system to avoid get-

ting stuck either in a retry loop or just waiting for a timeout, we add a circuit breaker in between. When the dependencies become not available, clients will behave more predictably. They will fail rapidly and consistently, as opposed to just hanging and timing out after some time.

This idea only works if the client is built in a way that can handle failures. When the circuit breaker trips, calls are guaranteed to fail fast. The circuit breaker does not remove the exception, it just makes it happen faster. Instead of waiting for 10 minutes and then raising the exception, when the circuit breaker trips, clients need to deal with the exception right away.

Clients need a strategy to tolerate unavailable services. They can disable part of the user interface, but keep the main user workflow available. Maybe the call was to get some optional information about a minor subcomponent of the user interface. If it takes time before you realize you are never going to get the answer, you are going to block the whole system for nothing. Instead if you can just hide or disable the failing component, but show everything else immediately, that's a much better experience for the user.

Canary Call

How to avoid crashing all recipients of a poisoned request?

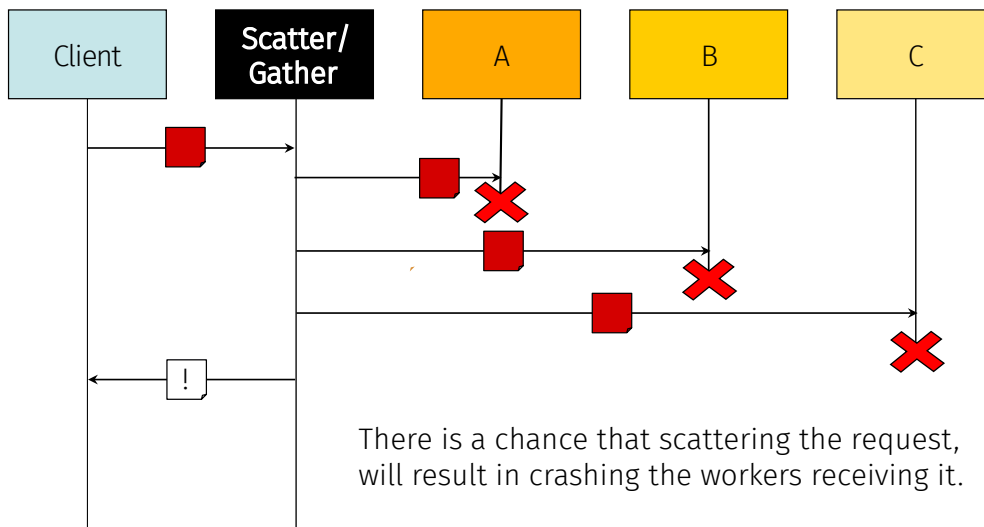
use an heuristic to evaluate the request

Try the potentially dangerous requests on one recipient and scatter the request only if it survives

The Canary call is a useful pattern when there is a chance that when you send a request, the recipient may crash directly as a consequence of your request. We call this a poisoned request.

The idea of the canary call is that you do not forward suspicious requests to everyone. But you first observe how one behaves when processing the request. If you get more confidence that this is not a poisoned request, then you can broadcast it. First try the request with one recipient and scatter it to all the others only if the first recipient survives. We use the Canary in the coal mine metaphor, if this particular recipient has a problem and then you can expect that everybody else will also have the same problem. So we are in the context for this is the scatter gather scenario in which just naively forwarding the request is going to crash all of your workers. And the goal is to avoid a potential high recovery cost.

Canary Call



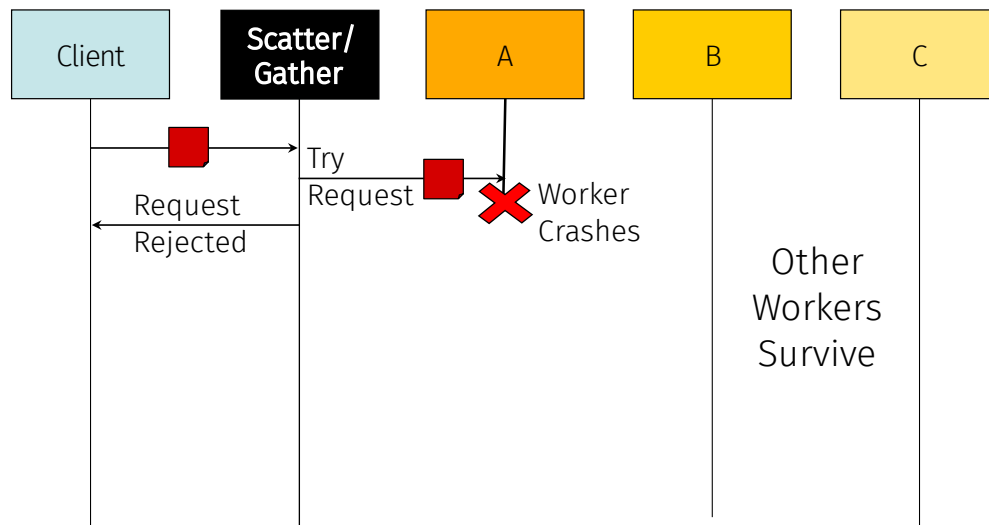
We illustrate the problem in the context of the scatter/gather pattern. You are not only sending the request to one component, but you send it to many components at the same time. For example, with a master worker architecture, what if there is something wrong with the input data and after partitioning it, you send the chunks out to a cluster with thousands of parallel workers and you crash all of them? To recover, you will need to reboot the cluster. It will take a long time before the system is available again. You didn't just lose one node, you lost everything, so all other requests got stuck in the queue.

What we can do is first try the request on one worker. And if it crashes, we don't forward it to the others, we just reject it.

This is a simple way to evaluate how dangerous is the request, with the advantages that if the request is rejected after one worker crashed, the other workers survive to process another set of requests. If the request is successful on one worker, we can send it to everybody else.

This is a pattern that we can apply in the context of a scalable architecture in which we want to spread out requests across multiple elements, and we want to try to protect these elements, prevent them from crashing when they receive a bad request.

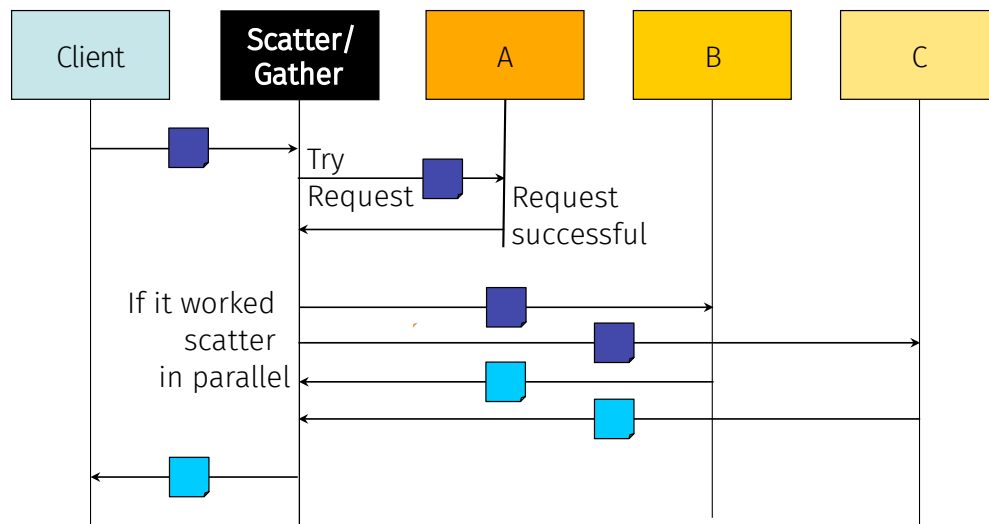
Canary Call



The first request is sent before all others to check if it would harm the worker receiving it (canary request).

If the canary request crashes the worker, it is not forwarded to the others which survive the attack.

Canary Call



The first request is sent before all others to check if it would harm the worker receiving it (canary request).

The decision on whether all other requests should be scattered depends on an heuristics: continue if the canary request is successful.

Canary Call

- Performance/Robustness Trade-Off:
 - Decreased Performance: the scatter phase waits for the canary call to succeed (response time doubles)
 - Increase Robustness: most workers survive a poisonous call, which would have failed anyway
- Apply when worker recovery is expensive, or when there are thousands of workers involved
- Heuristic: Failed canary calls are not necessarily poisonous

If we introduce this pattern, what's the advantage in terms of failures? Only one element will fail as opposed to every element.

In the successful case, when no failure occurs, what is the disadvantage? What do we lose? Compared to the original case in which we just risk to directly forward the request to all the elements, it takes more time.

Canary call requires to trade off between: How many of our workers we want to save? How much longer will it take before we can send the final response?

First we have to try out the request in one place, so this will take some time. If the request is valid, we can work in parallel as we scatter it out. We have just duplicated, in the best case scenario, the time that it takes to service the request. Since we have to wait first to see if it works and then we have to process it in the other places. It now takes twice as long.

Canary call is not something to be introduced without being aware of its performance cost. If you want to protect the availability of your cluster, you will just make every request run twice as slow.

You want to apply it when recovering a worker is very expensive. Or you have so many that could potentially fail that it will severely impact your data center. For example, if you send a request to modify the server configuration or install a security patch. Would you fire off such request to all of your servers without first upgrading one and observing whether it remains stable for a little while? If you lose half a data center, then it takes time before you can recover it and you may not have any backup while you are doing so.

If a request fails one worker, is it a good indication to predict that all workers will fail? How can you be sure that a crash in one place means that it is was poisonous request? Or was that machine going to fail anyway? That's difficult to tell, the pattern relies on a heuristic decision. You try, you observe and then you draw an inference about what may happen elsewhere.

What's interesting is that today there are many tools to make predictions. Machines can learn. Canary call is applicable also with more advanced heuristic to validate requests. If too many requests are discarded, clients may complain. If poisonous requests

do not always crash a worker, they may make it past the filter and wreck havoc on your data center. A request could be perfectly valid, but it triggers a bug in the recipient and until the recipient is not fixed, the request should be stopped from propagating.

In the simplest case, you can always try to run it somewhere and see what happens. If you can anticipate and detect malicious requests without actually running them, that's even better.

Redundancy

High Availability is achieved by introducing redundancy for critical components (primary and at least one backup)

$$\text{Cost of redundancy} < (1 - \text{Availability}) * \text{Cost of Unavailability}$$

Fail-over is the process of automatically switching from the primary to a backup instance, which is standing by:

- **Hot**, ready to begin working immediately
- **Cold**, needs to be initialized before it can take over

Load Balancing also helps to increase availability when faulty workers are pulled from rotation

If you are going to fly on an airplane and something goes wrong with one of the engines. Would you rather be flying on an airplane with four engines or an airplane that only has one engine?

Redundancy is about paying the price to increase the reliability of your architecture by dedicating enough spare resources so that in case something goes wrong you can still land safely.

If you cut costs in the name of being lean, then you will probably sacrifice the minimum level of redundancy that will make it possible for you to survive when something goes wrong. It may be a good idea to have some spare parts. To invest in cash reserves. To anticipate expected or unexpected problems in the future. Keep a few extra masks around the house for the next wave.

When we design a software architecture, we can decide to introduce redundancy. That means that we will make multiple copies, replicas, instances at different levels of granularity. The entire system could be deployed in different data centers. Individual components will be installed and be running in different containers, hopefully running on different physical hosts, installed across different data centers. Or the same interface could be re-implemented by different independent development teams to avoid replicating bugs.

Redundancy will have a cost, but these costs will be less than the penalty that you will need to pay in case something goes wrong.

If, when something goes wrong, there is no problem – after all users are always glad to take some unplanned vacations while you rebuild their server, restore the database from backups, and will just start happily working again when the problem is resolved.

But if you cannot afford that, then it's better to invest up front to introduce enough redundancy. It will pay for itself, or even be cheaper when things go wrong, but your company workflows are not disrupted thanks to such redundant components: when one

of them fails, we can still use the others. If one of the engines of the airplane doesn't work, then we can still fly because we have the others.

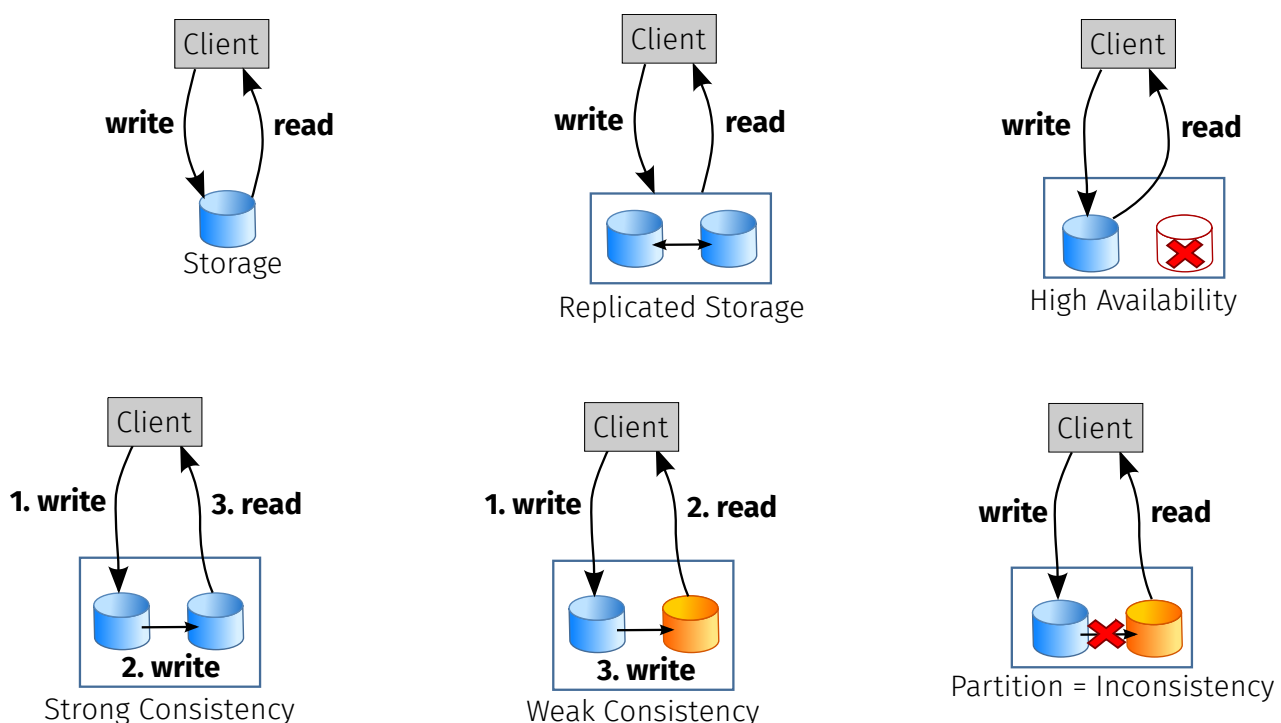
To minimize disruption, in some cases you will keep these backup components hot. This means that as soon as something stops, you can immediately continue by using the back up. We say that the failover is instantaneous.

Alternatively, you have to start from a cold backup, which takes longer to initialize and bring up to speed. If the hardware is not running to save energy, you have to power up the hardware. This takes time. Then you have to start the software and make sure it has the latest version of the data.

Cold standby means that it's ready to go, but you have to switch it on and wait for it to come online. If it's hot there is a higher cost to keep it running, but the advantage is that fail over is supposed to happen instantaneously.

You can also use redundancy in combination with load balancing strategies that we have discussed earlier. With load balancing you introduce redundancy for the purpose of scalability because you need more capacity. As a consequence, if one of the workers becomes problematic, you remove it from the pool. You stop sending work to this particular failed component. The rest of the system can survive because you have already the other workers which are still available.

State Replication



It's easy to build a redundant architecture if you have stateless components. Trivial: you can start as many copies as you want. Quickly initialize the component so that it can receive messages and process them without affecting its state, because there is none. So we're not going to talk about that.

What we will focus on is: how do we make redundant architecture out of stateful components? In other words, how do we keep replicated state synchronized or consistent?

That's the main challenge.

When you have stateful components, you make multiple copies, each represented with the usual cylinder. When there is no replication, life is easy because we can update the information atomically. There is only one place where this information is stored and that means that whenever we change it, it becomes immediately available for reading it.

After we introduce replication, to help in case one of the copies is lost, we have one or more backups. From a high level perspective it should look the same. Clients can interact with the component, change its state, read the updated made by other clients. Ideally, there should not be any outside impact due to the internal replication. Clients shouldn't notice that there are now multiple replicas that need to be kept synchronized. Also they should not notice when we lose one copy and fail over to the other one. We want to keep the same interface for the stateful component and just deploy it in a redundant configuration, which can tolerate the loss of some replicas.

However, imagine there are two copies; then we have two scenarios. You write into one copy. And then you synchronize with the other before reading. The first step will be that you send an update. You write something into the system. One copy transitions to the new state, then the replica follows, and then you read. Since both copies synchronized with the other, when you read it doesn't matter where you read from, because they're all in the same state. We call this situation strong consistency. We guarantee that no matter how many replicas you have, the information that you write can be read from any of the copies. It's like an ideal scenario, because you write something and then you can actually read it back. There is a cost: before you can read it, you have to propagate the update to all replicas. You have to synchronize the update, and this takes time.

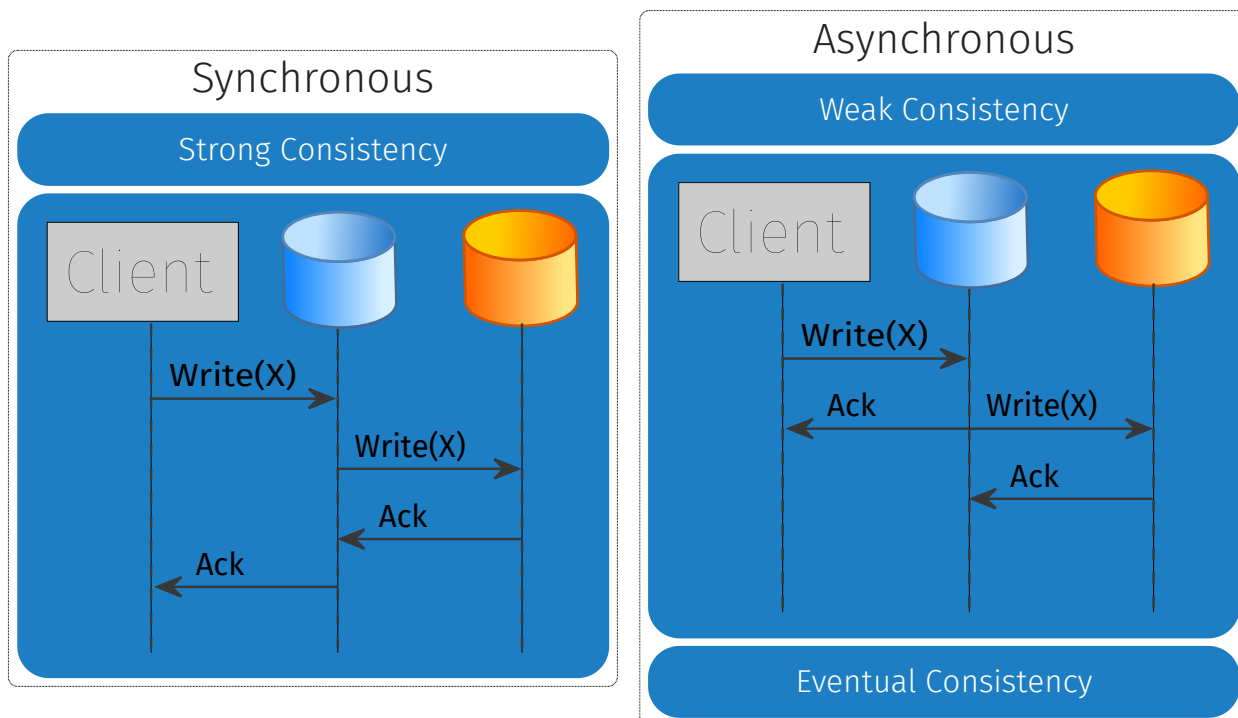
The other option does not give such guarantee. You write into one copy, but when you read, you may get the latest value or not. It can happen that you read before the synchronization has happened. You read from a stale replica. This is what we call the weak consistency. There is a risk that if you introduce replication, you not always able to see the changes that you make immediately. There will be a delay before all replicas have transitioned to the latest version.

As we have defined availability as: how long are you willing to wait? In some failure scenarios the replication delay becomes very long, maybe it becomes impossible for the two copies to synchronize because we have a partition between them. A network partition just means that the two sides cannot talk to each other, and therefore the synchronization will never happen. At least until the partition is resolved. Even if the replicas cannot synchronize, we can still write on one side and read from the other. Even if we retry, we keep reading the old data because it doesn't get synchronized because of the partition. As a consequence, we observe an inconsistency between the different replicas.

This just a very high level view into what can happen when we try to replicate stateful components. This problem occurs no matter whether they use a databases or just keep their state in memory. As soon as we make another instance and deploy it somewhere, there will be a second copy of the component state, which will suffer from this issue.

The advantage is that when something fails, you don't lose the data. You can use the other copy. In this scenario, you have the main copy and the backup. You have two copies, so you can survive if one fails. When you lost a copy, you're back into the non replicated scenario. So your goal is to avoid that, since you only one more failure away from losing everything. If something fail, you need to spawn a new replica as soon as possible. Depending on how big is the amount of state that you have to replicate, it will take time. Failures strike fast, disk crashes occur rapidly, while creating a new backup - like if you need to copy several terabytes - will take a significant amount of time.

Which kind of replication?



What is the difference between synchronous and asynchronous replication? Here is a more detailed view about what we have just discussed about strong and weak consistency.

When you are doing synchronous replication, you write some information into one of the replicas, then you try to send the copy to the other replica. You wait for the replica to confirm that the update it's actually been stored, and only then you tell the client that the write operation has completed. If you do this, you get strong consistency as a consequence.

Since the client has to wait for all the replicas to agree that the new state is the one that the client is writing, you have to wait until all replicas agree. This means that writes takes as long as the slowest replica. If replicas are scattered around the globe in different data centers, there will be a high latency before they confirm and you have to wait for the slowest replica to confirm before you acknowledge the client. That's the price to pay if you want to guarantee a strong consistency using synchronous replication. Unless all replicas agree, the write didn't happen.

If the client is in a hurry, you can speed up the write with the asynchronous replication alternative. You update one copy. As soon as you send out the update to the other replicas you already – very optimistically – tell the client that the information has been stored. So what does this mean? Well, it has been stored in one place, but the synchronization with the others is still in progress. It can happen that the client receives the acknowledgement, and for some reason wants to read the value. There is now a race between the client read and the synchronization. It is possible that the read will not return the latest value because it reads from replica that is not yet up to date.

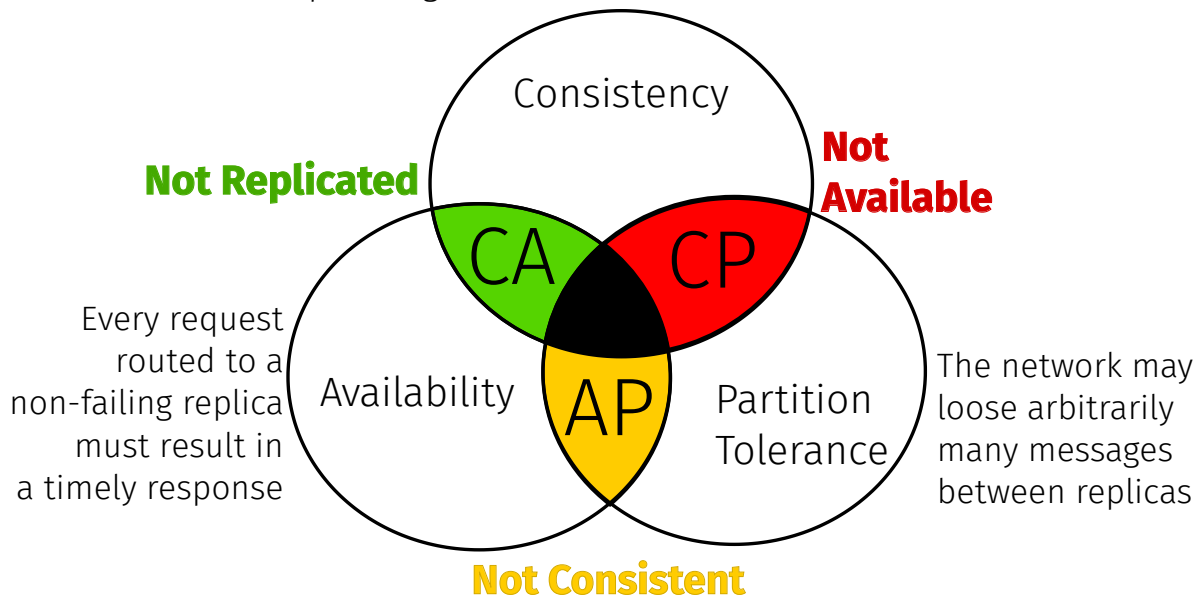
We call it asynchronous because the synchronization happens in the background. The main reason for doing that is to make the writes from the client faster. We give the opportunity for the replicas to catch up at their own pace. As a consequence, we have weak consistency since there is a point in time in which replicas do not agree. Since the

synchronization is in progress, eventually the replicas will actually get to the new state. That's why it's also known as eventual consistency.

We have seen two simple replication protocols, with different behavior when something fails. If one waits for all the replicas to reach the new state, the information will be redundantly stored before the write is confirmed. If it is not possible to duplicate the data, the write will fail. With the optimistic approach, the synchronization happens asynchronously. If you lose the only up-to-date copy, before you managed to propagate it to the replica, you can have information loss after informing the client that the write was successful. It is possible that as you acknowledge the client you discover that there is a partition which doesn't let the write through to the replica. This means that until the partition is resolved, there will be only one replica which has the latest state. If you lose it, there is no backup.

CAP Theorem

All replicas agree on the latest version of their state



A distributed replicated system cannot have both Strong Consistency and Availability

What we have discussed is also called the CAP theorem. When you have a stateful component that is replicated across a distributed system, there is a limit that stated by this theorem in which you cannot have both availability and strong consistency and also you can be tolerant of partitions.

What does it mean when you have only two out of three properties? As you can see from the Venn diagram, we have consistency if all of the replicas agree on the latest version of the state. We have availability if when we send a request we get a timely answer. That's the quality we have been talking about today.

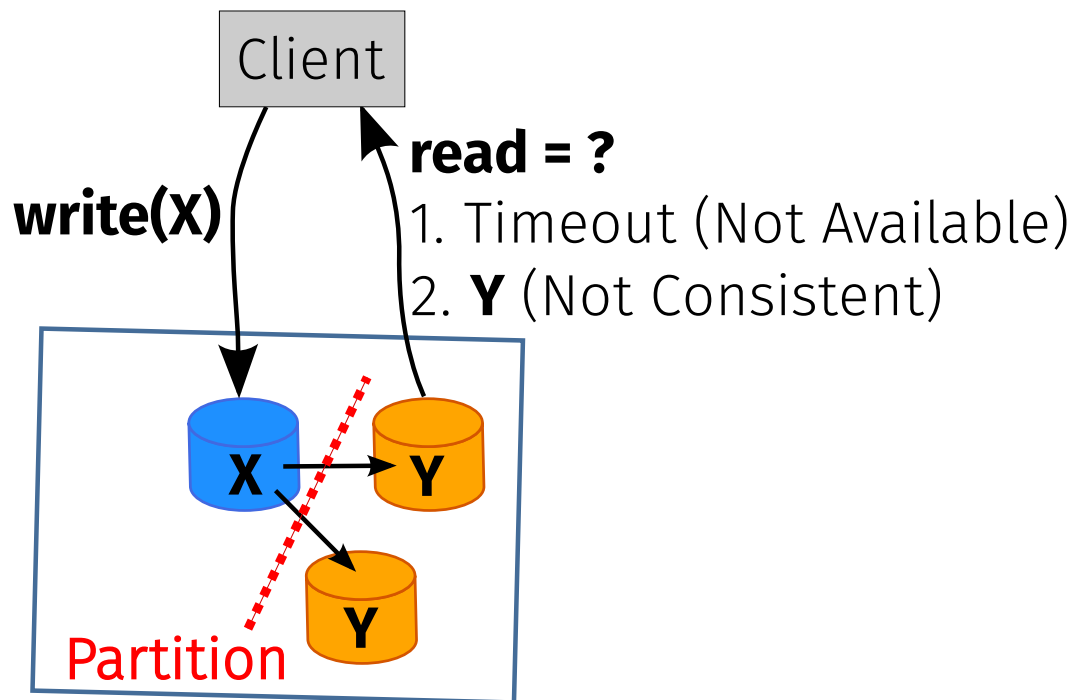
What does it mean to survive partitions between the different replicas? This is true if the network is fully reliable and will never lose any of those synchronization messages.

The CAP theorem states that in the intersection of all of these three properties is not possible. You have to choose which pair of properties you want to have. Which one do you prefer? Availability or Consistency? Let's explore the trade-off.

If you want to design an architecture in which you want to provide consistency, therefore, when a partition occur, you will have to sacrifice availability. If you prefer to have availability, you will need to sacrifice the consistency.

Why do I say that only these two cases are interesting. Because the only kind of system that is partition tolerant is a centralized system that is not replicated. Only in those systems you can have both availability and consistency. As soon as you replicate, you bring in distribution, decentralization, and then you have to choose. A real network will fail. You will have partitions and therefore your architecture can either be not available or not consistent.

CAP Theorem Proof



This is probably the only proof that we do in the lecture.

Let's go back to the previous scenario in which you have a client that is writing and updating one of the replicas. So what can happen? The client wants to read? Check that the state transition was successful.

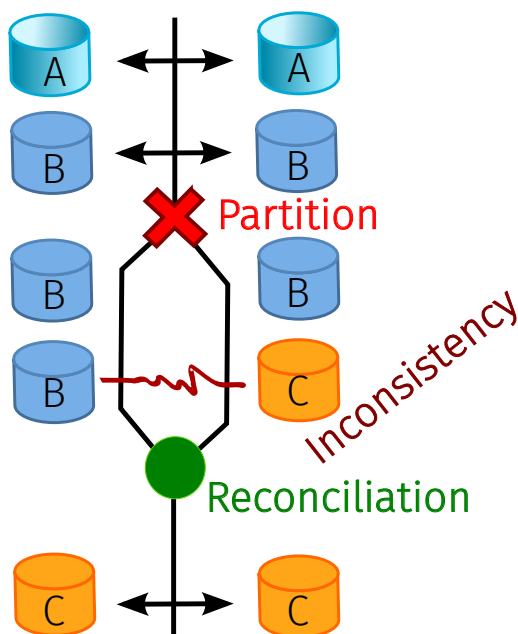
Let's say that we are not partition tolerant and a partition just happened. So it is not possible to synchronize the replicas. Because of this problem, what could the read result be?

Since the client wrote X but it reads Y from an out-of-date replica, the result is not consistent. Since synchronization was not possible, the result is stale.

What if we wait to give just enough time for the partition to resolve itself and the synchronization to work. Is the client going to be willing to wait that long? You mean the client is going to time out and give up reading? But that means that the client will say that the read was not available. Well, at least the client didn't read an inconsistent value. Actually, it didn't read any value at all.

These are the only two cases when we have a partition. Either immediately read an obsolete result (available, but inconsistent) or wait forever for the result to arrive, in practice give up waiting (not available). Also in this case, if the client receives a result, it will be the consistent one.

Eventual Consistency



Some applications prioritize availability at the expense of consistency.

During the recovery from a partition, conflicts between multiple inconsistent states need to be resolved manually or automatically (e.g., last writer wins).

The data semantics within some applications may tolerate this for short periods of time.

While availability and consistency cannot be guaranteed at the time of the partition, if we choose availability we can still recover the consistency, eventually, after the partition has been resolved.

If one waits long enough, the replicas will get synchronized. Eventually, after rebooting your WiFi router you will be online again.

Eventual consistency helps to keep your system available. You just don't promise that the latest state will be visible everywhere, but you just say that if you wait long enough it will get across.

What does this mean? Here we have a replicated system that is in a certain state. Then we have the partition. The two replicas can no longer talk to each other. Is the system automatically inconsistent? Not yet. It's just a partition, but the two copies are still the same, so we're still consistent. There's still a chance that if you fix the network, you reconnect while you never lost the consistency.

The inconsistency is only introduced when you do the update. If you change the state only one of the replicas would be updated and the other one will become obsolete and it would be inconsistent.

Is the inconsistency visible at that time? Not unless you can reach all the replicas and compare their state. After solving the partition, you have to decide which replica has the latest state. How to solve this conflict? It is possible to associate a version counter or a timestamp and see which has the most recent version.

You may have experienced something similar once you tried to pull changes into your code and there was a merge conflict. Sometimes the merge works automatically, sometimes you need to resolve it manually.

If the inconsistency is small enough, if the amount of change that you have introduced

is not too big, if the time in which some replica was offline is not too long, then it's possible to reconcile it in a finite amount of time. If there is a replica cut off for a number of years, then you have a lot of work ahead if you want to reconcile all the changes accumulated for such a long time. Still, despite the fact that you were inconsistent for awhile, eventually you can go back to a consistent state. It all depends on how long "eventually" means for you.

Event Sourcing

How to reconstruct past states?

How to synchronize replicated states?

Keep an explicit representation of the state history

Log all events that led to state transitions so that they can be replayed.

Speaking of accumulating changes, this is what the Event Sourcing pattern is about. This is relevant for highly available, redundant architectures, in which there are replicated stateful components. The idea is not to store only a snapshot of their latest state, but to store the history of all the changes from which the current state can be obtained.

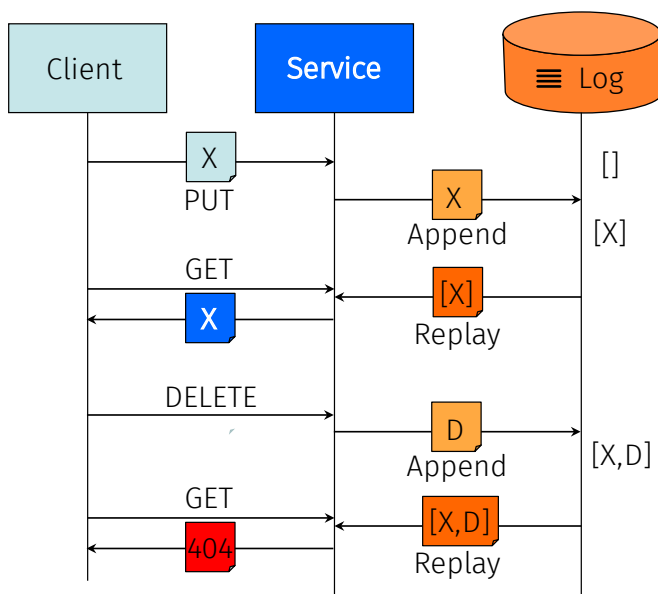
This helps when you have to deal with conflict resolution, since you don't just have two inconsistent states to compare, but you can see how you got there by tracking the sequence of changes.

Even though your source code files conflict, you can resolve the conflict because developers have been editing different lines and you can merge them in the right sequence.

Instead of storing the latest value, keep a history of all the changes that you've done. Since events trigger state transitions, we can use the term event to represent a change. If still remember the previous state before the event occurred, the change that was applied, then you can obtain the new state.

To store all events, we use a log structure. If you want to take it to the extreme, you can use a blockchain, but you don't have to. You just do it in a simple log in which you have a time stamp and information associated with the change event. If timestamps are not possible, it is enough to keep an ordering between the events. If you log all the input that you gave to a stateful and deterministic component, given a known initial state, you can always replay the log and stop at a certain point to reconstruct the past history of the state of this particular component.

Event Sourcing



Track individual changes as opposed to complete snapshots

For example, if we make a change to the state of a certain service and we use event sourcing, it means that we need to keep track of state changing operations by appending them to the event log.

A PUT operation, which means that you want to write this value X as the next state does not simply set the current state to X, but logs the event (the state is now X) into the log.

The log is represented as an array of all the changes that have occurred so far. It's easy to reconstruct what is the current state, because there is only one entry in the log. The initial state was empty and after the PUT operation we enter into the state X, which is what the GET (a read operation) returns.

Here comes another change: DELETE should reset the state to the initial one. We're not simply going to do it directly. We're not emptying the state, we're actually storing the fact that we have a request to delete the state. And the log grows to include this event (represented as D).

When we read the current state again, we will replay the log. The state does not become X anymore because of the delete event. As a consequence, we show to the outside that after you delete something then the information is gone. However, in the log it is still present. If you wanted to undo the effect of some of the changes, you can still time travel along the past history.

Event Sourcing

Performance Tradeoff

- Append-only log of immutable events (fast write)
- Replay log to compute the latest state snapshot from known initial state (slow read)
- Cheaper to store (or transfer) a log of state transitions vs. the complete history of state snapshots
- Cheaper to store one snapshot than the log of all state transitions to reproduce it

When you use event sourcing to implement state storage, writes are fast because it's enough to append them to the log, while reads can be slow because you need to replay the whole log, unless you keep a cached version of the latest state.

The amount of data that you have to transfer is the critical bottleneck when you introduce replication in your stateful components. Event sourcing can help to build an eventually consistent, replicated solution in which you store the component state across multiple copies. As opposed to transfer entire and potentially large snapshots every time a few bits change, it is more efficient to send a stream of changes to keep the copies consistent.

In general, event sourcing makes sense when the cost of logging individual state changes is smaller than storing a snapshot of the whole dataset. Consider introducing it if the data is large and there are few small changes. It may be inefficient if the data is small and there are lots of changes, so the log size grows above the size of just taking a snapshot.

How long is the history of state transitions? If you keep your stateful component running for a long time with lots of clients invoking it, then you have a long history and it will become cheaper just to store the current state and read it as opposed to replaying all the log to compute the latest state.

There will be a point that even with event sourcing, you will take anyway a snapshot and only keep the changes from the snapshots onwards. A good time to do so is when all replicas are in sync.

By flattening a large log of state changes into a snapshot of the current state you can save space but lose the ability to time travel.

Event Sourcing

Advantages

- Record audit trail (what happened when, who did what)
- Synchronization (replay from the last known event to catch up)
- Conflict Resolution (make sure events are played in the same order)
- Rollback (replay up to a given event)
- Reproduce bugs (replay production log on a test/upgraded system)

In addition to the synchronization between replicas, and the fine-grained conflict resolution, event sourcing has more advantages.

You can establish some kind of a legal audit trail. You track not only which are all the changes that happened, and when they took place, but also who is responsible for them. This can be very useful also from a business point of view to store all the history. Also implementing undo is just a matter of replaying the log and stopping before reaching the final event.

It's also great for testing and reproducing bugs. The log is a valuable source of input for your tests. Replay the log and check if different versions of the component end up in the same state. After you make a new release of the system, you can spot compatibility issues or regressions by simply applying the event log and comparing the obtained state.

Event Sourcing

Disadvantages

- Storage size (logs tend to grow unbounded, take a snapshot from time to time)
- Serializability (what's the right order of the events?)
- Consistency (what if events get lost?)
- Evolvability (how to version events in the log? how to migrate old logs so that they can be played with newer clients?)

Disadvantage include the problem of an unbounded growth of the event log. The more state transitions happen, the more events need to be listed in the log. If you want to join the mining effort, you get the privilege of downloading and storing your own copy of all bitcoin transactions since the origin block.

You have to be careful about the order in which you're logging things, especially if you have multiple logs in a decentralized architecture. It's not trivial to know what is the right sequence of events unless there is a centralized log or you are really careful with the clock synchronization.

It is also critical not to lose any of the events. Given an incomplete log, there is no guarantee that replaying it will result in the same state. That's why blockchain miners dedicate so much energy to continuously re-checking all the hashes to make sure that the log didn't get tampered with.

What about if the format of the events changes? Can a new release of the component still play back events logged from an older version of the component? There is an issue of migrating the logs that due to their historical nature spanning across multiple releases of the component is more complex as opposed to just migrating the snapshots. We will see more about software evolution in the next lecture.

References

- Werner Vogels, [Eventually Consistent](#), ACM Queue, Dec 2008
- Eric Brewer, CAP Twelve Years Later: How the "Rules" Have Changed, Computer, 45(2):23-29, Feb. 2012
- Niall Richard Murphy, Betsy Beyer, Chris Jones, Jennifer Petoff (Eds.), [Site Reliability Engineering](#): How Google Runs Production Systems, O'Reilly, April 2016, ISBN 978-1491929124
- Michael T. Nygard, Release It! Design and Deploy Production-Ready Software, 2018, ISBN 978-1-68050-239-8
- Ali Basiri, Nora Jones, Aaron Blohowiak, Lorin Hochstein, Casey Rosenthal, Chaos Engineering, O'Reilly, August 2017, ISBN 9781491988459
- Guy Pardon, Cesare Pautasso, Olaf Zimmermann: [Consistent Disaster Recovery for Microservices: the BAC Theorem](#). In: IEEE Cloud Computing, 5(1): 49-59, 2018
- [Is it down right now?](#)

Software Architecture

Flexibility and Microservices

12

Contents

- API Evolution: To break or not to break
- Layers of defense against change
- Extensibility and Plugins
- Microservices
- Feature Toggles
- Splitting the Monolith